

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ВГРАДЕНО ХРАНИЛИЩЕ ЗА RDF ТРИПЛЕТИ
С МЕХАНИЗЪМ ЗА ИЗПЪЛНЕНИЕ НА
ЗАЯВКИ НА SPARQL 1.1**

КУРСОВ ПРОЕКТ

по дисциплина „Семантичен уеб“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р инж. Аделина Алексиева-Петрова

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА РЕШЕНИЯТА	2
1.1. Информационен модел, синтаксиси и семантика	2
1.2. Обосновка на направения избор	3
II. ПРОЕКТИРАНЕ И ПРОГРАМНА РЕАЛИЗАЦИЯ	4
2.1. Слоеве, речник и индекси	4
2.2. План, ред на съединенията и оператори	4
2.3. Реализация и поведение при грешка	6
III. ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА И РЕЗУЛТАТИ	7
3.1. Коректност	7
3.2. Условия за валидност на измерването	7
3.3. Резултати: зареждане и изпълнение на заявки	8
3.4. Анализ	9
ЗАКЛЮЧЕНИЕ	10
ЛИТЕРАТУРА	11
ПРИЛОЖЕНИЯ	12

УВОД

Семантичният уеб описва информацията във вид, който машина може да свърже, вместо само да покаже. Единицата на записа е *триплетът*: подлог, сказуемо и допълнение. Моделът RDF задава абстрактния синтаксис на този запис [1], а SPARQL задава как над множество триплетите се формулира заявка [2]. Двете спецификации са нормативни, но между тях и работеща програма стои инженерна задача, която те съзнателно не решават: как триплетите се съхраняват, индексират и обхождат така, че заявката да завърши за приемливо време.

Повечето реализации решават тази задача като сървър, тоест отделен процес, мрежов протокол и администриране. Има случаи, в които това е излишно: настолно приложение или конвейер за обработка на данни, който трябва да зададе няколко заявки над локален набор и да продължи. За такъв случай е подходящо *вградено* хранилище, тоест библиотека, която работи в процеса на самото приложение.

Целта е да се проектира и реализира вградено хранилище за RDF триплетите с механизъм за изпълнение на заявки на SPARQL 1.1 и да се измери поведението му. Задачите са четири: обосновка на решенията по всяко отворено място (кодиране на термините, набор от индекси, вид на анализаторите, стратегия на планировчика); проектиране на слоевата архитектура; реализация на C++20 с изграждане чрез CMake; и методика за проверка и измерване, следвана от анализ.

Обхватът покрива четене на Turtle и на N-Triples, записване на N-Triples, съхранение с речниково кодиране и пермутирани индекси, анализ на подмножество на SPARQL 1.1 до алгебрично дърво, планиране на реда на съединенията и изпълнение над индексите. Извеждането по RDFS е отделен слой в режим на материализация. Извън обхвата остават протоколът SPARQL през HTTP, SPARQL Update, именуваните графи, разпределеното изпълнение, запазването на хранилището между стартирания и разсъждаването по OWL 2. Точната граница на поддържаното подмножество е в приложение Б.

Исходният текст, тестовите и измерителната програма са публични: <https://github.com/Alex-Tsvetanov/trident>.

I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА РЕШЕНИЯТА

1.1. Информационен модел, синтаксиси и семантика

Класическият уеб свързва документи с хипервръзка без тип: тя казва, че два документа са свързани, но не и как. Семантичният уеб описва ресурсите и дава на всяка връзка собствен идентификатор и значение. Множество триплети образува насочен етикетан граф, в който допълнението на един триплет може да бъде подлог на друг, и така отделно съставени описания се свързват без обща схема. RDF 1.1 въвежда трите вида термини, тоест IRI, литерал и празен възел, и разделя абстрактния модел от конкретните му записи [1]; затова анализаторите са сменяеми входове към един вътрешен модел. Turtle е компактният четим запис [3], а N-Triples е негово подмножество с по един пълен триплет на ред [4].

```
1 @prefix ex:    <http://example.org/> .
2 @prefix rdfs:  <http://www.w3.org/2000/01/rdf-schema#> .
3
4 ex:alice a ex:Author ; ex:name "Alice" ; ex:age 34 ;
5           ex:knows ex:bob, ex:carol .
6
7 ex:Author    rdfs:subClassOf    ex:Person .
8 ex:mainAuthor rdfs:subPropertyOf ex:author .
```

Листинг 1: Част от data/small.ttl

Синтаксисът определя кои записи са допустими, но не и какво следва от тях. Семантиката на RDF задава кога един граф следва от друг [5], а RDF Schema надгражда с речник за класове, свойства, области и обхвати [6]: от последните два реда на примера следва, че `ex:alice` е и `ex:Person`, и че всеки триплет по `ex:mainAuthor` важи и по `ex:author`. Езикът OWL 2 добавя ограничения върху свойствата, кардиналности и дизюнктни класове, и с това променя изчислителната сложност на извеждането [7]; проектът реализира RDFS като отделен слой и разглежда OWL 2 само описателно. SPARQL 1.1 дефинира базови графови шаблони, незадължителни части, обединения, филтри, агрегация, подзаявки и пътища по свойства, а също и алгебра, към която текстът се превежда преди изпълнение [2]; тази алгебра е границата между анализатора и изпълнението, приета за такава и тук.

Литературата за съхранение на триплети се съсредоточава върху един въпрос:

базовият графов шаблон изисква търсене по различни комбинации от свързани и свободни позиции, а един ред на сортиране обслужва добре само част от тях. Hexastore предлага пълния набор от шест пермутации, срещу шесткратно повторение на данните [8], а RDF-3X показва, че при речниково кодиране подходът е приложим, и добавя планировчик, който избира реда на съединенията по статистики върху самите индекси [9].

1.2. Обосновка на направения избор

Таблица 1. Разгледани алтернативи и направен избор

Решение	Разгледани варианти	Избор	Водещ довод
Език	C++20, Rust, Java	C++20	вграждане без отделна среда за изпълнение
Термини	низове в триплета, речниково кодиране	речниково кодиране	фиксиран размер, евтини сравнения
Индекси	един, три, шест пермутации	три пермутации	покрытие на всички шаблони без шесткратен обем
Носител	памет, външен склад, отразени файлове	редици в паметта	същите алгоритми, без платформено зависим вход и изход
Анализатори	генератор, рекурсивно спускане	рекурсивно спускане	по-добри съобщения, без стъпка в изграждането
Планировчик	фиксирано правило, оценка по статистики	оценка по статистики	устойчив спрямо реда на изписване

Изборите са обобщени в (Табл. 1), а два от тях имат нужда от повече от един ред довод. **Носителят:** между подредени редици в паметта и същите редици във файл разликата е само откъде идват байтовете. Избрана е паметта, защото отразяването във файлове изисква платформено зависим вход и изход, без да променя алгоритмите; ограничението е записано като такова, а именно че наборът трябва да се събира в паметта. **Планировчикът:** при подредени индекси броят на триpletите, напасващи един шаблон, е дължината на диапазона от двоичното търсене, тоест пресмята се точно за $O(\log n)$.

II. ПРОЕКТИРАНЕ И ПРОГРАМНА РЕАЛИЗАЦИЯ

2.1. Слоеве, речник и индекси

Всеки слой зависи само от слоевете под себе си (Фиг. 1). Ключът на речника е записът на термина в N-Triples, тъй като е каноничен: термин, написан с префикс или изцяло, дава един ключ и се вписва веднъж. Идентификаторът е 64 бита: старшите 8 носят вида на термина, а младшите 56 поредния номер, така че изпълнението отговаря на `isIRI`, `isLiteral` и `isBlank` без да чете термина. Празните възли получават обхват от името на файла, за да не могат два документа да се сблъскат.



Фигура 1. Слоеве. Ляво е пътят при зареждане, дясно е пътят на заявката

След кодиране триплетът е три числа, тоест 24 байта. Всеки индекс е непрекъсната редица от такива записи, подредена лексикографски по своята пермутация: SPO с ред на сравнение (0, 1, 2), POS с (1, 2, 0) и OSP с (2, 0, 1). Изборът на точно три пермутации стои върху следното твърдение: за всяка от осемте комбинации от свързани и свободни позиции съществува индекс, в който свързаните позиции образуват префикс (Табл. 2). Проверката му е записана като тест, а не оставена като разсъждение.

Търсенето по шаблон е двоично търсене на границите на префикса, следвано от последователно четене на диапазона. Последната колона е също толкова съществена: обхождането произвежда триплетите подредени по позицията след префикса, а тази безплатна подредба е единственото условие, при което сливането е допустимо, тоест в план с лява дълбочина само за първата двойка.

2.2. План, ред на съединенията и оператори

Планировчикът работи само върху алгебричното дърво и не познава синтаксиса, което позволява двете части да се тестват поотделно. Преобразуването към план прави

Таблица 2. Избор на индекс според свързаните позиции на шаблона

Свързани позиции	Индекс	Префикс	Подредба на изхода
няма	SPO	0	по подлог
подлог	SPO	1	по сказуемо
сказуемо	POS	1	по допълнение
допълнение	OSP	1	по подлог
подлог, сказуемо	SPO	2	по допълнение
сказуемо, допълнение	POS	2	по подлог
подлог, допълнение	OSP	2	по сказуемо
и трите	SPO	3	единичен резултат

три неща: дава на всяка променлива номер на позиция, така че свързването по време на изпълнение е достъп по индекс в масив, а не търсене по име; кодира константите през речника, при което непозната константа прави шаблона празен още преди каквото и да е обхождане; и фиксира реда на съединенията.

Под *мощност* на шаблон се разбира броят на триплетите, които го напасват. Тя се пресмята точно за $O(\log n)$, което премахва цял клас грешки от разминаване между оценка и действителност. Редът се избира лакомо: първи е шаблонът с най-малка мощност, а после най-малкият измежду онези, които споделят променлива с вече избраните, защото шаблон без обща променлива дава декартово произведение. За да може ползата от планирането да бъде измерена, а не предположена, планировчикът приема флаг, който запазва реда на изписване.

Изпълнението е верига от итератори с два метода: отваряне, което получава вече направените свързвания от обхващащия оператор, и извличане на следващото решение. Този договор е причината съединяването чрез вложени цикли да бъде съединяване по индекс: вътрешният оператор получава свързванията на външния и стеснява диапазона си с тях, вместо да чете всичко и да отсява. Операторите са обхождане по индекс, две съединявания (по индекс и чрез сливане), незадължително съединяване, обединение, филтър, проекция, премахване на повторения, подреждане, групиране с агрегация, ограничение и отнемстване. Материализират само подреждането и агрегацията; останалите работят поточно, затова ограничението на броя решения спира работата рано.

Извеждането по RDFS е материализация: правилата се прилагат след зареждане и

получените триплети се вмъкват в индексите. Реализирани са **rdfs2** и **rdfs3** (област и обхват дават тип на подлога и на допълнението), **rdfs5** и **rdfs11** (транзитивност на подсвойството и на подкласа), **rdfs7** (триплет по подсвойство важи и по надсвойството) и **rdfs9** (ресурс от подклас е и от надкласа) [6]. Прилагат се до неподвижна точка, защото се подхранват едно друго: обхватът дава тип, а подкласът след това разширява този тип. Броят на обиколките е ограничен отгоре, за да не може дефект да доведе до безкраен цикъл вместо до съобщение за грешка. Извеждането по време на заявка, аксиоматичните триплети и правилата за контейнери са съзнателно извън обхвата.

2.3. Реализация и поведение при грешка

Проектът е организиран в `include/trident/` с публичните заглавни файлове, `src/` с реализацията по един файл на слой, `tools/` с трите изпълними програми и `tests/` с тестовете; обемът по модули е в приложение А. Библиотеката е статична и няма нито една външна зависимост: публично хранилище, чието изграждане изисква мениджър на пакети или изтегляне по време на конфигуриране, страничен наблюдател не изгражда.

Възлите на алгебричното дърво са обединени в един `std::variant`, защото над дървото се извършват няколко обхождания и нито едно от тях не е естествена отговорност на възела. Обратно, операторите на изпълнението са йерархия с виртуални методи, защото там има точно едно поведение и то се различава при всеки оператор. Двата анализатора споделят четец, който води ред и колона, затова съобщенията за грешка имат еднакъв вид; N-Triples е същият анализатор с флаг, който отхвърля добавеното от Turtle. Наборът от данни се генерира с детерминиран линеен конгруентен генератор, защото набор, който се различава между машините, не може да служи за сравнение.

Зареждането е двустъпково: триплетите се натрупват в междинен буфер и влизат в хранилището едва след като целият документ е прочетен, така че документ с грешка по средата не оставя частично заредени данни. Конструкция извън подмножеството се отхвърля с изрично съобщение, защото пренебрегната конструкция би дала заявка, която означава нещо различно от написаното.

III. ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА И РЕЗУЛТАТИ

3.1. Коректност

Синтаксисът се проверява със съставен за целта набор от 101 случая: 41 за Turtle (26 положителни и 15 отрицателни), 16 за N-Triples (7 и 9) и 44 за SPARQL (26 и 18). Всички преминават. Отрицателният случай се проверява два пъти: веднъж, че изобщо е отхвърлен, и втори път, че отказът носи ред, колона и текст, защото анализатор, който казва само „синтактична грешка“, не върши работа. Осемнадесетте отрицателни случая за SPARQL са конструкциите извън подмножеството, тоест проверени граници, а не дефекти. Наборът е съставен в хранилището, а не изтеглен, защото тестовите набори на W3C [10] са изтегляне, а проектът трябва да се изгражда без мрежов достъп; затова тези числа не бива да се четат като резултат от нормативния набор.

Семантиката се проверява върху малък граф от 40 триплета, чиито отговори са преброени на ръка от текста на файла, а не отчетени от реализацията. Пълният набор съдържа 125 случая в 10 групи в CTest и всичките преминават. Отделно се проверяват и двете твърдения на Раздел II: че за всяка от осемте комбинации избраният индекс има префикс с дължина, равна на броя свързани позиции, и че планираният ред, наивният ред, сливането и вложените цикли дават едно множество решения.

3.2. Условия за валидност на измерването

Наборът се генерира от хранилището и описва библиографски граф: статии със заглавие, година, конференция, тема, брой цитирания и автори; автори с име, месторабота и показател, като само част от тях имат домашна страница, за да има какво да показва незадължителното съединение. Схемната част добавя подкласове, подсвойство, област и обхват, а работното натоварване е осемте заявки от приложение В.

Измерванията са проведени на AMD Ryzen 5 3600 с 6 ядра и 15,9 GiB памет, под Windows 11 Pro N, с g++ 15.2.0 (MinGW-w64) в режим CMAKE_BUILD_TYPE=Release, CMake 4.3.2 и Ninja 1.13.2. Всяка конфигурация се изпълнява 15 пъти, първите две се отхвърлят заради загряването на разпределителя на памет, и се докладва медианата. Броят на решенията се отпечатва до всяко време: ред, в който броевете не съвпадат, не е сравнение на скорост. Машината е работна, а не изолиран стенд, затова наборът е повторен три пъти; суровият изход е в docs/measurements/, а таблиците са от второто

изпълнение.

3.3. Резултати: зареждане и изпълнение на заявки

Таблица 3. Зареждане, изграждане на индексите и обем

Статии	Триплети	Вход, В	Анализ, ms	Триплети/s	Индекси, ms	Обем, KiB
1 000	10 462	301 893	12,2	858 239	2,0	1 827
5 000	52 395	1 528 499	59,4	882 778	11,1	7 024
20 000	209 202	6 185 046	261,4	800 498	57,4	28 032
50 000	522 910	15 546 537	504,6	1 036 328	114,5	72 123

Броят на различните термини в речника е 2 135, 7 614, 27 565 и 67 501. Времето за анализ расте линейно (Табл. 3), а пропускателната способност остава от порядъка на 800 000 до 1 000 000 триплета за секунда, без спад при петдесеткратно нарастване на входа. Изграждането на трите индекса е около една пета от времето за анализ, тоест сортиранията не са водещият разход, а обемът е около 141 байта на триплет, което включва трите копия по 24 байта и речника. Разсейването е най-голямо именно тук: при 20 000 статии способността е 474 237, 800 498 и 905 522 триплета за секунда, тоест почти два пъти разлика, и величината следва да се чете като порядък.

Таблица 4. Медианно време при включен и при изключен планировчик, 209 202 триплета

Заявка	Планирано, ms	Наивно, ms	Отношение	Решения
V1 един шаблон	1,375	1,400	1,02	20 000
V2 звезда от два шаблона	6,822	6,831	1,00	20 000
V3 верига от четири шаблона	0,028	12,705	447,4	47
V4 верига от пет шаблона	0,028	12,833	465,0	0
V5 филтър по число	6,172	6,122	0,99	3 188
V6 незадължително съединение	1,352	1,348	1,00	4 000
V7 групиране с брояч	2,613	2,918	1,12	200
V8 подреждане на цялото	13,623	13,894	1,02	20 000

Колоната с броя решения (Табл. 4) е условие за валидност: двата режима връщат едно и също множество при всяка заявка. Нулата при V4 е действителният отговор, а не пропуснатото измерване, тъй като авторите от избраната организация нямат статия в

избраната конференция; редът остава, защото времето до празния отговор се различава 465 пъти.

Стратегията на съединяване беше сравнена върху заявката с два шаблона със свързано казуемо, споделящи допълнението си, тоест единствената форма, при която сливането е допустимо. Двете връщат по 99 996 решения; сливането отнема 10,313, 8,995 и 9,149 ms, а вложените цикли по индекс 10,112, 10,049 и 10,042 ms. При мощност на водещия шаблон 4, 5 и 11 заявката отнема 0,022, 0,022 и 0,028 ms и връща 29, 30 и 47 решения, тоест времето расте с мощността, а не с размера на набора. Материализацията по RDFS извежда 68 398 нови триплета за две обиколки до неподвижна точка, при което наборът става 277 600, а обемът расте от 28 032 на 38 985 KiB за 168,6, 312,3 и 157,5 ms.

3.4. Анализ

Планирането има смисъл там, където има какво да се планира. При един или два шаблона двата реда се различават в границите на шума, а при вериги от четири и пет шаблона отношението е между 447 и 465 пъти. Причината е пряка: при наивен ред първият шаблон е `?p ex:title ?t` с мощност 20 000, а при планиран ред първи е `?a ex:affiliation ex:org3` с мощност 11, тоест междинният резултат тръгва с три порядъка по-малък. Разходът за планиране е под една стотна от милисекундата, а точната мощност се оказва и по-проста, и по-надеждна от оценка: структурата на индекса вече е статистиката.

Стратегията на съединяване не се оказва водеща. Разликата между сливане и вложени цикли е в границите на разсейването. Това противоречи на очакването при проектирането и е записано такова, каквото е; числата допускат обяснението, че при план с лява дълбочина сливането е приложимо само за първата двойка шаблони.

Какво не е измерено. Сравнение с Apache Jena [11] и RDFLib [12] беше предвидено, но не е проведено: и двете изискват изтегляне и среда за изпълнение, каквито на машината не са налични. Число, което не е измерено, няма място в отчет; работното натоварване и генераторът обаче са записани така, че сравнението да може да бъде проведено по-късно. Не са измерени върховата резидентна памет на процеса, поведението при набор, който не се събира в паметта, и цената на извеждане по време на заявка, тъй като този режим не е реализиран. Времената преди двете промени в изпълнението също не се докладват: междинните състояния не са запазени. Изводите важат за една машина.

ЗАКЛЮЧЕНИЕ

Проектът решава задачата за съхранение на RDF триплети и изпълнение на заявки над тях във вид, подходящ за вграждане в приложение. Архитектурата разделя синтаксиса, съхранението, анализа, планирането и изпълнението на слоеве с тесни граници, което позволява всеки да бъде проверяван поотделно, а изграждането става с компилатор за C++20 и CMake, без външни зависимости.

Предимства. Речниковото кодиране и подредените пермутирани индекси свеждат съединяването до обхождане на диапазони и премахват низовите сравнения от горещия път. Трите пермутации покриват всичките осем комбинации от свързани позиции, което е проверено, а не прието. Точната мощност се получава от самия индекс за $O(\log n)$, така че планировчикът работи от истински числа.

Ограничения. Индексите се държат изцяло в паметта и се изграждат наново при всяко отваряне, а трикратното повторение струва 72 от измерените 141 байта на триплет. Поддържаното подмножество на SPARQL не покрива именувани графи, подзаявки, пътища по свойства, BIND, VALUES и HAVING, а извеждането по RDFS е само в режим на материализация. Сравнението с утвърдени реализации не е проведено и причината е записана в Раздел III, а не заобиколена с предположения.

Изводи. Планирането на реда на съединенията си заслужава точно при веригите от три и повече шаблона, където отношението между непланирания и планирания ред е между 447 и 465 пъти, докато при един или два шаблона то е в границите на шума; разходът за планиране е под една стотна от милисекундата, откъдето следва, че планировчик от този вид няма смисъл да се прави изключваем в производствен код. Стратегията на съединяване се оказва второстепенна, което противоречи на очакването при проектирането. За разглеждания случай на употреба измерените величини са в приемливи граници: около 800 000 до 1 000 000 триплета за секунда при зареждане, около 141 байта на триплет и под три стотни от милисекундата за селективна заявка над 209 202 триплета. Заявките от порядъка на десет милисекунди връщат двадесет хиляди решения, тоест разходът е в резултата, а не в достъпа до данните.

По-нататъшна работа. Отразяване на индексите във файлове; компресия; извеждане по RDFS по време на заявка; именувани графи; и сравнението с външни реализации, за което работното натоварване вече е готово.

ЛИТЕРАТУРА

- [1] R. Cyganiak, D. Wood, and M. Lanthaler, “RDF 1.1 concepts and abstract syntax.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/rdf11-concepts/>.
- [2] S. Harris and A. Seaborne, “SPARQL 1.1 query language.” W3C Recommendation, Mar. 2013. <https://www.w3.org/TR/sparql11-query/>.
- [3] E. Prud’hommeaux and G. Carothers, “RDF 1.1 Turtle: Terse RDF triple language.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/turtle/>.
- [4] G. Carothers and A. Seaborne, “RDF 1.1 N-Triples: A line-based syntax for an RDF graph.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/n-triples/>.
- [5] P. J. Hayes and P. F. Patel-Schneider, “RDF 1.1 semantics.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/rdf11-mt/>.
- [6] D. Brickley and R. V. Guha, “RDF Schema 1.1.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/rdf-schema/>.
- [7] W3C OWL Working Group, “OWL 2 Web Ontology Language document overview (second edition).” W3C Recommendation, Dec. 2012. <https://www.w3.org/TR/owl2-overview/>.
- [8] C. Weiss, P. Karras, and A. Bernstein, “Hexastore: sextuple indexing for semantic web data management,” *Proceedings of the VLDB Endowment*, vol. 1, no. 1, pp. 1008–1019, 2008.
- [9] T. Neumann and G. Weikum, “The RDF-3X engine for scalable management of RDF data,” *The VLDB Journal*, vol. 19, no. 1, pp. 91–113, 2010.
- [10] W3C SPARQL Working Group, “SPARQL 1.1 test suite,” 2013. <https://www.w3.org/2009/sparql/docs/tests/>.
- [11] The Apache Software Foundation, “Apache Jena documentation.” <https://jena.apache.org/documentation/>.
- [12] RDFLib Team, “RDFLib documentation.” <https://rdflib.readthedocs.io/>.

ПРИЛОЖЕНИЯ

Приложение А. Изходен текст и стартиране

Целият изходен текст, тестовете, измерителната програма и този отчет са в публичното хранилище <https://github.com/Alex-Tsvetanov/trident>. Той не е вмъкнат тук, защото се чете по-добре в редактор, отколкото на хартия. Всеки ред в (Табл. 5) е двойка заглавен файл в `include/trident/` и файл с реализация в `src/`.

Таблица 5. Модули на реализацията, отговорност и обем

Модул	Отговорност	Редове
<code>term,</code> <code>dictionary</code> <code>store</code>	термин, 64 битов идентификатор, двупосочно съпоставяне	272
<code>lexer, turtle</code>	кодирани триплетите, трите индекса, търсене по префикс, точни мощности	258
<code>sparql_ast,</code> <code>sparql_parser</code> <code>expression</code>	четец на знаци и термини, анализатор на Turtle и N-Triples	869
<code>planner</code>	алгебрично дърво и рекурсивното спускане към него	992
<code>exec</code> <code>rdfs</code> <code>query,</code> <code>generator</code>	изчисляване на изрази във филтри и подредбата за сортиране	523
<code>tools/</code> <code>tests/</code>	номериране на променливите, ред на съединенията, избор на стратегия	310
	операторите на механизма за изпълнение	883
	материализация по правилата на RDFS	195
	лицева функция и детерминиран генератор на набора	331
	<code>trident_demo</code> , <code>trident_bench</code> , <code>trident_cli</code>	596
	собствена рамка и десет тестови групи, 125 случая	1 906
Общо		7 135

Изисква се компилатор за C++20 и CMake 3.20 или по-нов. Външни зависимости няма и по време на конфигуриране не се тегли нищо. `cctest` докладва 10 записа и 125 отделни случая, а `trident_demo` зарежда генерирания набор, изпълнява десет заявки и за всяка отпечатва алгебричното дърво, избрания план, броя решения и изтеклото време.

```
1 cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
2 cmake --build build
3 cctest --test-dir build --output-on-failure
```

```

4 build/trident_demo
5 build/trident_bench --repeats 15

```

Листинг 2: Изграждане, тестове, демонстрация и измервания

Приложение Б. Поддържано подмножество на SPARQL

Разпознава се: PREFIX и BASE; SELECT с изброени променливи или *; DISTINCT и REDUCED; базови графови шаблони със съкратени списъци и с ключовата дума а; литерали с езиков етикет и с тип; празни възли във вид _:етикет и []; вложени групи; OPTIONAL; UNION; FILTER; GROUP BY по променливи; COUNT, SUM, MIN, MAX, AVG и SAMPLE, със и без DISTINCT; ORDER BY, LIMIT и OFFSET. В изразите: ||, &&, шестте сравнения, четирите аритметични операции, унарни ! и -, и 25 функции, сред които BOUND, STR, DATATYPE, isIRI, STRLEN, CONTAINS, SUBSTR, REGEX и COALESCE.

Отхвърля се с изрична грешка: ASK, CONSTRUCT, DESCRIBE, GRAPH, SERVICE, MINUS, BIND, VALUES, HAVING, подзаявки, пътища по свойства, GROUP BY по израз и всяко име на функция извън списъка. Всеки от тези случаи има отрицателен запис в набора от синтактични случаи, тоест границата на подмножеството е проверявана, а не само описана.

Приложение В. Заявки, използвани при измерванията

Всички заявки започват с PREFIX ex: <http://example.org/>, пропуснат по-долу.

```

1 B1 SELECT ?p ?t WHERE { ?p ex:title ?t }
2 B2 SELECT ?t ?y WHERE { ?p ex:title ?t . ?p ex:year ?y }
3 B3 SELECT ?t ?n WHERE { ?p ex:title ?t . ?p ex:mainAuthor ?a .
4                          ?a ex:name ?n . ?a ex:affiliation ex:org3 }
5 B4 SELECT ?t ?n WHERE { ?p ex:title ?t . ?p ex:mainAuthor ?a .
6                          ?a ex:name ?n . ?a ex:affiliation ex:org3 .
7                          ?p ex:venue ex:venue2 }
8 B5 SELECT ?p WHERE { ?p ex:citationCount ?c FILTER (?c > 100) }
9 B6 SELECT ?n ?h WHERE { ?a a ex:Author . ?a ex:name ?n
10                        OPTIONAL { ?a ex:homepage ?h } }
11 B7 SELECT ?v (COUNT(?p) AS ?n) WHERE { ?p ex:venue ?v } GROUP BY ?v
12 B8 SELECT ?p WHERE { ?p ex:citationCount ?c } ORDER BY DESC(?c)
13
14 Стратегия на съединяване:
15 SELECT ?a ?first ?second WHERE { ?first ex:mainAuthor ?a .
16                                   ?second ex:author ?a }

```

Листинг 3: Работно натоварване